

Assignment-8.1

Ravula Ajay

2303A52370

Batch-44

Task 1: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Code:

```
def is_strong_password(password):
    if len(password) < 8:
        return False
    if " " in password:
        return False

    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False

    for ch in password:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        else:
            has_special = True

    return has_upper and has_lower and has_digit and has_special

# AI-generated assert test cases
assert is_strong_password("Abcd@123") is True
assert is_strong_password("abcd123") is False
assert is_strong_password("ABCD@1234") is False
assert is_strong_password("Abcdef1!") is True
assert is_strong_password("Abc 123!") is False
assert is_strong_password("A1!bcdef") is True
```

Task 2: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Code:

```
def classify_number(n):
    for _ in range(1):
        if not isinstance(n, (int, float)) or isinstance(n, bool):
            return "Invalid"
        if n == 0:
            return "Zero"
        if n > 0:
            return "Positive"
        return "Negative"
```

```
# AI-generated assert test cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(-1) == "Negative"
assert classify_number(1) == "Positive"
assert classify_number("10") == "Invalid"
assert classify_number(None) == "Invalid"
```

Task 3: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

```
def is_anagram(str1, str2):
    def normalize(value):
        chars = []
        for ch in value.lower():
            if ch.isalnum():
                chars.append(ch)
        chars.sort()
        return chars
    return normalize(str1) == normalize(str2)
```

```
# AI-generated assert test cases
assert is_anagram("listen", "silent") is True
assert is_anagram("hello", "world") is False
assert is_anagram("Dormitory", "Dirty Room") is True
assert is_anagram("", "") is True
assert is_anagram("A man, a plan, a canal: Panama!", "A Panama canal: a plan, a man!") is True
assert is_anagram("same", "same") is True
```

Task 4: Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Code:

```
class Inventory:
    def __init__(self):
        self._stock = {}

    def add_item(self, name, quantity):
        if not isinstance(quantity, int) or quantity < 0:
            raise ValueError("quantity must be a non-negative integer")
        self._stock[name] = self._stock.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        if not isinstance(quantity, int) or quantity < 0:
            raise ValueError("quantity must be a non-negative integer")
        current = self._stock.get(name, 0)
        if quantity > current:
            raise ValueError("insufficient stock")
        self._stock[name] = current - quantity

    def get_stock(self, name):
        return self._stock.get(name, 0)

# AI-generated assert test cases
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
inv.remove_item("Book", 0)
assert inv.get_stock("Book") == 3
```

Task 5: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

Code:

```
def validate_and_format_date(date_str):
    if not isinstance(date_str, str):
        return "Invalid Date"

    parts = date_str.split("/")
    if len(parts) != 3:
        return "Invalid Date"

    month_str, day_str, year_str = parts
    if not (month_str.isdigit() and day_str.isdigit() and year_str.isdigit()):
        return "Invalid Date"

    if len(month_str) != 2 or len(day_str) != 2 or len(year_str) != 4:
        return "Invalid Date"

    month = int(month_str)
    day = int(day_str)
    year = int(year_str)

    if month < 1 or month > 12:
        return "Invalid Date"

    days_in_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        days_in_month[1] = 29

    if day < 1 or day > days_in_month[month - 1]:
        return "Invalid Date"

    return f"{year_str}-{month_str}-{day_str}"
```

```
# AI-generated assert test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("13/01/2023") == "Invalid Date"
assert validate_and_format_date("02/29/2024") == "2024-02-29"
assert validate_and_format_date("2/29/2024") == "Invalid Date"
```